EDA: T1: Introducción a la Programación Funcional

1. Programación funcional

1.1. Programación imperativa vs programación funcional

-En programación imperativa los cómputos manipulan **variables mutables** que representan la memoria o el estado del computador.

- Los **algoritmos modifican** las **variables** hasta obtener el resultado deseado.

-La programación funcional pura es un estilo de programación basado en **cómputos** a través de **funciones matemáticas** (o puras). Un **programa** es una **función** que:

- Toma datos a través de sus **argumentos**.
- Realiza cálculos con estos datos a través de **expresiones**.
- Devuelve nuevos datos (**resultados** del cómputo).
- **¡Nada cambia!**: No existe el concepto de variable mutable típico de la programación imperativa.

1.2. Funciones puras e impuras

-Una función matemática siempre devuelve el mismo resultado para los mismos valores de sus argumentos:

$x = y \Rightarrow f(x) = f(y)$ (Regla de Leibniz)

-Esto conduce a la propiedad de **transparencia referencial**: una misma expresión denota siempre el mismo valor, sea cual sea el punto del programa o el contexto en que aparezca.

-Simplifica el **razonamiento** sobre las **propiedades** de los **programas** (en particular, la corrección).

-En el caso de las **funciones impuras** en cada **ejecución** del programa suele **modificarse** una **variable** de forma que la siguiente ejecución es distinta.

- Podemos 'calcular' con programas al igual que en matemáticas 'calculamos' con expresiones.

1.3. La programación con lenguajes funcionales

-La programación funcional consiste en escribir **programas** (funciones) **combinando** otras **funciones**.

-En los lenguajes funcionales puros:

- Solamente podemos definir **funciones puras**.
- Se satisface la **transparencia referencial**. Ej: Haskell, Miranda, Clean, etc.

-En los lenguajes funcionales impuros:

- Se permite definir **funciones impuras**. Ej: ML, F#, Scheme, etc.

1.4. Ventajas de la programación funcional pura

-Optimización: el compilador puede realizar **transformaciones sofisticadas** con objeto de **optimizar** el **comportamiento**.

-Paralelismo: **partes** de una **expresión** pueden **evaluarse** en **cualquier orden** sin alterar el resultado; incluso **varios procesadores** pueden **intervenir** en la evaluación de la misma expresión.

- La Programación Funcional Pura es intrínsecamente paralela.
- La Programación Imperativa es intrínsecamente secuencial.

-Memoization: los resultados producidos por las llamadas a una función pueden ser reutilizados (recordados) en sucesivas llamadas con los mismos valores de los argumentos.

2. Introducción a Haskell

-Es un **Lenguaje Funcional Puro Estándar** cuyas principales características son la **pureza funcional**, la **tipificación estática** y que es **perezoso** (lazy).

2.1. Identificadores

-Haskell **distingue mayúsculas** de **minúsculas** (*case sensitive*).

-Los **identificadores** (**nombres**) de funciones y argumentos:

- Deben **empezar** con una **letra minúscula** o el **carácter _ (*underscore*)** y éste puede ir **seguido** de **letras** (mayúsculas o minúsculas), **dígitos**, **tildes** (') o **_**.
- Los **nombres** de **tipos** deben empezar con una **letra mayúscula**. Ej: Int, Integer, Bool, Float, Double, etc.

-Los nombres de operadores (Ej. válidos: +, *, -, !!, <=, ==>, +: , etc.):

- Pueden contener **uno** o **varios símbolos**.
- El **primero no** puede ser **dos puntos** (:).

2.2. Tipos numéricos

-Int un **subconjunto acotado** de los **números enteros**: operaciones rápidas; posibilidad de *overflow* o *underflow.* (Valor mínimo: $-2^{31}-1$ y máximo: $2^{31}-1$).

-Integer es el **conjunto** '**completo'** de los **enteros**.

-Float es un subconjunto de los reales en simple precisión.

-Double es un subconjunto de los reales en doble precisión.

2.3. Booleanos

-Bool tiene solamente dos valores: True y False.

- Conjunción lógica (and): (&&) :: Bool -> Bool -> Bool
- Disyunción lógica (or): (||) :: Bool -> Bool -> Bool.
- Negación lógica: not :: Bool -> Bool

- Hay evaluación en cortocircuito:

- En el operador lógico **"&&" (AND)** si la primera condición es falsa, el resultado final de la expresión será falso sin importar el valor de las demás condiciones: las condiciones posteriores no se evaluarán.
- En el operador lógico **"||" (OR)**, si la primera condición es verdadera, el resultado final de la expresión será verdadero sin importar el valor de las demás condiciones: las condiciones posteriores no se evaluarán.

2.4. Caracteres

- Char es el conjunto de **caracteres Unicode**: 'a', 'b', …, 'z', 'A', 'B', …, 'Z', '0', '1', … '9', '+', '?', etc.

- Muchas **funciones** sobre **caracteres** aparecen **definidas** en el **módulo** o **librería Data.Char**: para utilizarlo se importa (import Data.Char).

2.5. Parcialización (*currying*)

- Haskell usa notación parcializada (*curried*): para aplicar una función a sus argumentos:

- Escribimos el identificador de la función y a continuación los **argumentos** separados por **espacios**.
- El uso de paréntesis sólo será necesario para agrupar términos compuestos y modificar el efecto de las prioridades de las operaciones.

2.6. Condicionales

- Una **expresión condicional**, como cualquier otra expresión, puede formar parte del **cuerpo** de una **función**:

```
maxInteger :: Integer -> Integer -> Integer
maxInteger x y = if x >= y then x else y
```

2.7. Recursión

- El cuerpo de una función puede contener la función que define.

- El uso del tipo **Int** (subconjunto finito de enteros) causará problemas ya que el conjunto está acotado. El problema se resuelve **tomando Integer** (una **abstracción** del **conjunto** completo de los **enteros**).

2.8. GHC y GHCi

-**GHC** es el *Glasgow Haskell Compiler*, un compilador de alta calidad para Haskell.

-**GHCi** es el *Glasgow Haskell Compiler Interpreter*: con el uso del bucle Read-Eval-Print podemos evaluar programas Haskell.

2.9. Operadores

- El operador unario - es el único operador unario simbólico prefijo: -(1+2) -> -3

- El resto de operadores simbólicos son binarios y se usan directamente en forma infija:

- 1+2 -> 3.
- 1 + 2*3 -> 1+6
- (1+2)*3 -> 3*3
- 1 + -2 -> Precedence parsing error.
- 1 + (-2) -> -1

2.10. Nivel de precedencia

	Nivel	Operador predefinido
Máxima precedencia	10	Aplicación de una función
	9	. !!
	8	^ ^^ **
	7	* /
	6	+ -
	5	: ++
	4	== /= < <= > >=
	3	&&
	2	||
	1	>> >>= =<<
Mínima precedencia	0	$ $!

La aplicación de una función tiene precedencia máxima

La multiplicación * tiene mayor nivel de precedencia que la suma +

2.11. Asociatividad

- Esta característica permite resolver la ambigüedad en ausencia de paréntesis:

- Si un operador ◊ asocia a la izquierda:

  x1 ◊ x2 ◊ … xn = (…((x1 ◊ x2) ◊ x3) … ◊ xn).

- Si ◊ asocia a la derecha:

  x1 ◊ x2 ◊ … xn = (x1 ◊ … (xn-2 ◊(xn-1 ◊ xn))…)

- Si ◊ no es asociativo:

  x1 ◊ x2 ◊ … xn = Error de compilación: los paréntesis son necesarios.

Asociatividad	Operador
Derecha	^ ^^ ** : && || $ $!
Izquierda	* / + - >> >>= =<<
No asociativo	== /= < <= >= >

-Asociatividad vs Propiedad asociativa: son diferentes conceptos que no deben ser confundidos:

- La asociatividad es un atributo sintáctico de un operador en un lenguaje de programación.

- El **operador** ◊ verifica la **propiedad asociativa** en **sentido matemático** si satisface: ∀ x, y, z . (x ◊ y) ◊ z = x ◊ (y ◊ z).

2.12. Operadores y notación prefija

-Toda **función** de **dos argumentos** cuyo **identificador** sea **literal** puede ser utilizada en **forma infija** encerrándolo entre **acentos graves** (`).

-Ej: La función max :: Int -> Int se puede usar tanto como max 2 2 como 2 `max` 2.

-Recíprocamente, cualquier operador simbólico puede utilizarse en **forma prefija encerrándolo** entre **paréntesis.**

-Ejemplo: (+) 2 2 es equivalente a 2 + 2.

2.13. Evaluación de expresiones

-**Redex** (*reducible expression*): parte de una expresión que puede ser simplificada.

- **Reducción**: proceso de reescritura de un redex. Cada reducción es un paso de cómputo.

- **Forma Normal**: expresión sin redexes.

Existen dos tipo de orden a la hora de evaluar una expresión:

-Orden aplicativo: los **argumentos** son evaluados **antes** que la **función** (se reduce el redex más interno y más a la izquierda):

- **Pros**: los argumentos compuestos se evalúan una sola vez.
- **Contras**: pueden realizarse reducciones innecesarias y puede no conducir a la forma normal (no es normalizante).

-Orden normal: la **función** es aplicada **antes** de evaluar sus **argumentos** (se reduce el redex más externo y más a la izquierda):

- **Pros**: sólo se evalúan las expresiones necesarias y siempre conduce a la forma normal si ésta existe (es normalizante).
- **Contras**: puede repetirse la evaluación de un argumento compuesto.

-Haskell usa evaluación perezosa (*lazy*): combina las ventajas del orden normal y del aplicativo (similar al orden normal pero evita la reevaluación):

- Los argumentos son evaluados una sola vez y su valor común es compartido (*sharing*).
- Esta optimización es posible en lenguajes puros.

2.14. Tuplas

-Son **estructuras** de **datos heterogéneas**:

- Es una colección de valores (componentes).
- Cada componente puede tener un tipo distinto.

-**Sintaxis**:

- Los valores se separan por comas y se agrupan entre paréntesis.

-Funciones polimórficas: en caso de que a la hora de **definir** una **función** en lugar de especificar el tipo de la variable utilicemos una **letra minúscula** significa que puede ser de **cualquier tipo**. Ejemplos:

- fst :: (a,b) -> a // fst (x,y) = x.
- snd :: (a,b) -> b // snd (x,y) = y.

2.15. Sobrecarga y tipos numéricos

-Una **clase** de **tipos** es un **conjunto** de **tipos** que **comparten** alguna **operación**.

-**Num** es la clase predefinida para los tipos numéricos (enteros, flotantes, racionales, ...).

-Algunos **operadores** y **funciones** solamente tienen **significado** para **tipos numéricos**:

(+), (-), (*) :: (Num a) => a -> a -> a

-El **tipo sobrecargado** a debe ser una **instancia** de **Num**, es decir, debe ser un **tipo numérico** (debe leerse como: ∀ a ∈ Num . a -> a -> a).

-**Integral** es la clase que agrupa los tipos que representan números enteros.

-Define funciones específicas para la división entera (cociente y resto de la división entera):

div, mod :: (Integral a) => a -> a -> a

-**Fractional** es la clase que agrupa los tipos de números fraccionarios: flotantes y racionales (cocientes de enteros , ...)

(/) :: (Fractional a) => a -> a -> a

-**Ord** es la clase de los tipos para los cuales existe una relación de orden total; entre éstos están *Integer*, *Double*, *Char*, *Rational*, *Bool*, …

-Ciertas **operaciones** solo tienen sentido para las **instancias** de **Ord**:

(<), (<=), (>), (>=) :: (Ord a) => a -> a -> Bool

compare :: (Ord a) => a -> a -> Ordering

-data Ordering = LT | EQ | GT (Tipo enumerado predefinido: less than, equal y greater tan)

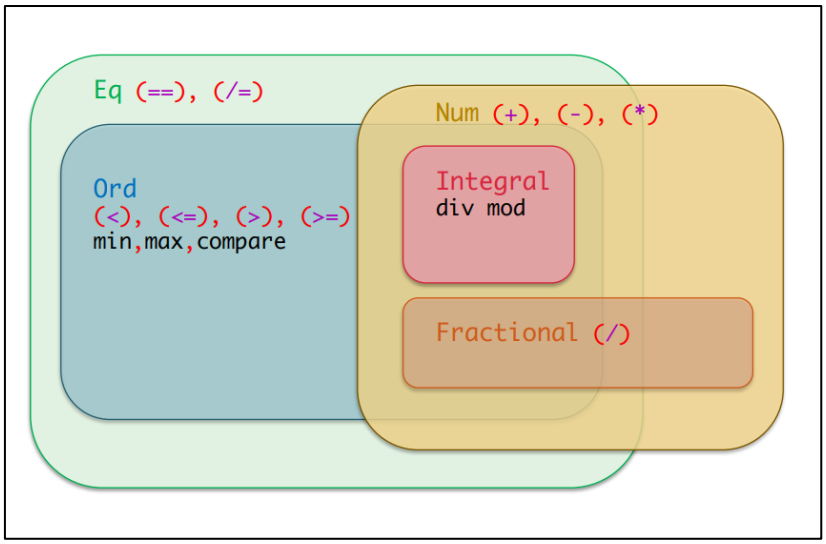
max, min :: (Ord a) => a -> a -> a

-**Eq** es la clase de los tipos para los cuales existe una **relación** de **igualdad**: entre éstos están *Integer*, *Double*, *Char*, *Rational*, *Bool*, …

(==), (/=) :: (Eq a) => a -> a -> Bool

2.16. Resumen de clases y sobrecarga

En el siguiente cuadro se resumen las clases principales y se muestra cuáles están dentro de otras así como las operaciones que cada una comparten:



2.17. Ecuaciones con guardas

-A menudo es más sencillo describir una **función** por **partes** (distintos casos):

```
sign :: (Eq a, Ord a, Num a) => a -> a
sign x  | x > 0 = 1
        | x < 0 = -1
        | x == 0 = 0
```

-Muy importante el **sangrado** (que todas las barras verticales estén en la misma columna) así como que después de cada | vaya un **espacio**.

-La definición anterior puede escribirse de **forma** más **corta** (**estilo preferente**):

```
sign :: (Ord a, Num a) => a -> a
sign x  | x > 0 = 1
        | x < 0 = -1
        | otherwise = 0
```

-**Otherwise** es una condición que siempre es cierta y será la que se tomará en caso de que fallen las anteriores.

-Las expresiones **condicionales** proporcionan una **alternativa** a las **guardas**.

2.18. Funciones parciales

-Hay funciones **definidas** solo **parcialmente** por lo que en los casos en los que **no** está **definida** se utiliza la función de **error**:

```
reciprocal :: (Eq a, Fractional a) => a -> a
reciprocal x  | x == 0 = error "reciprocal is undefined"
              | otherwise = 1 / x
```

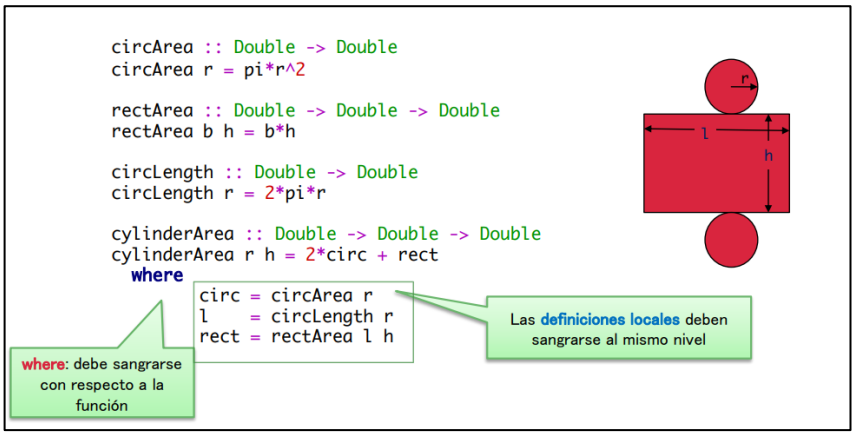
-La función **error** es una función **polimórfica**: error :: String -> a

-En caso de **ejecutar** la función **reciprocal** con el **0** se **devolvería** lo **siguiente**:

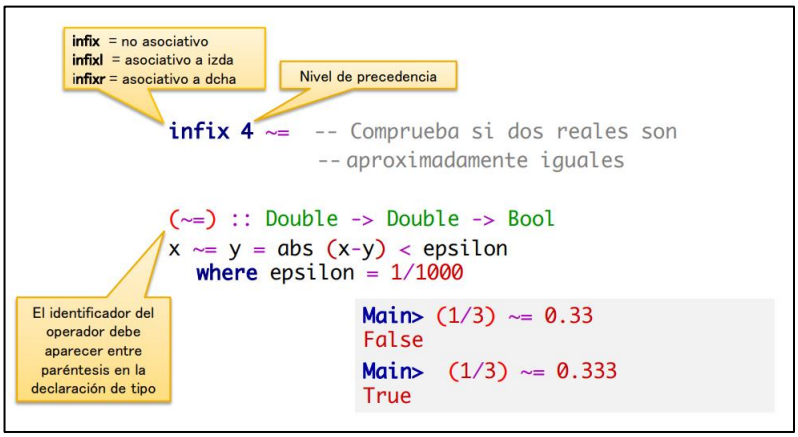
```
Main> reciprocal 0
*** Exception: reciprocal is undefined
```

2.19. Definiciones locales *where*

-La palabra reservada *where* permite definir funciones o variables locales, y debe aparecer al final de la definición:



2.20. Declaraciones de operadores



2.21. Pruebas con *QuickCheck*

-*Test.QuickCheck* es una **biblioteca** de **Haskell** que ayuda a **probar** los **programas**:

- Definimos **propiedades** que nuestros **programas** deben **cumplir**.
- ***Test.QuickCheck*** genera automáticamente **casos** de **prueba** y **verifica** las **propiedades** para esos casos.

-Propiedades en ***QuickCheck***:

antecedente ==> consecuente

- **Antecedente** y **consecuente** deben ser **expresiones booleanas**.
- Debe **leerse** como una **implicación lógica**: si el **antecedente** es **cierto** entonces el **consecuente** también debe **serlo**.
- ***QuickCheck*** solo realiza las **verificaciones** para los **casos** en los que el **antecedente** es **cierto**. Las pruebas generadas cuyo antecedente es falso son descartadas.
- Se puede usar *True* si **no** hay **antecedente**: *True ==> consecuente*.

2.22. Definiciones locales *let in*

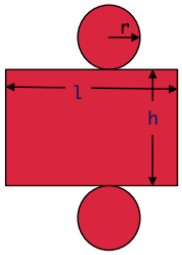
-Una **alternativa** para introducir **definiciones locales** es vía la construcción *let in*:

```
circArea :: Double -> Double
circArea r = pi*r^2

rectArea :: Double -> Double -> Double
rectArea b h = b*h

circLength :: Double -> Double
circLength r = 2*pi*r

cylinderArea :: Double -> Double -> Double
cylinderArea r h =
  let
    circ = circArea r
    l    = circLength r
    rect = rectArea l h
  in
    2*circ + rect
```



Las **definiciones locales** deben sangrarse al mismo nivel

Resultado de la expresión **let** ···**in**

2.23. Patrones en Haskell

- x: patrón **variable**, acepta cualquier valor y le da un nombre.

- (patrón_1, ..., patrón n): patrón **tupla** de **dimensión n**, permite imponer restricciones.

- n: patrón **constante**, solo acepta el valor literal de la constante.

- _ : patrón **subrayado**, acepta cualquier valor pero no le da nombre.